Taller Práctico: API

Objetivo

Desarrollar una API que permita gestionar empleados mediante operaciones CRUD, incluyendo la creación, consulta, actualización y eliminación de datos.

Descripción

Se desarrollo una API utilizando Node.js y Express que permite manipular datos de empleados almacenados en la memoria. La API responde a diferentes solicitudes HTTP para realizar operaciones CRUD.

Endpoints utilizados

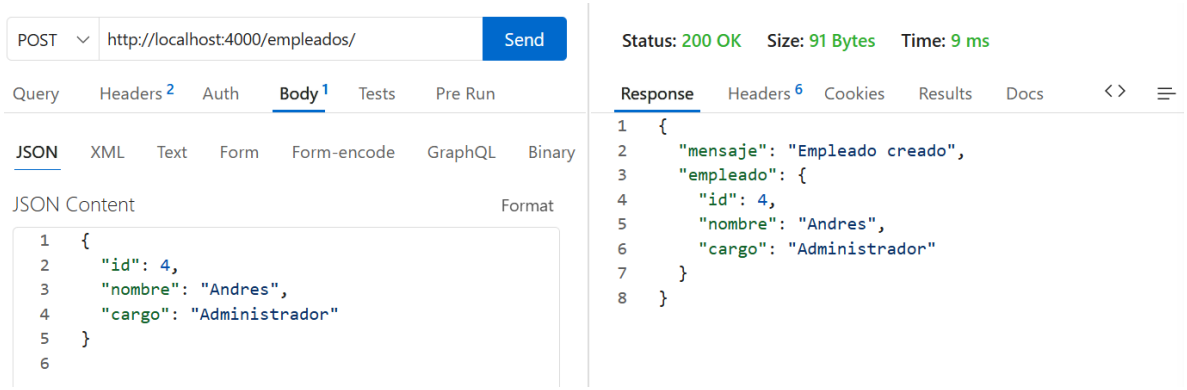
- POST/empleados ⟹ Crear empleado
- GET/empleados ⟹ Listar empleados
- GET/empleados/:id ⟹ Buscar empleado
- PUT/empleados/:id ⟹ Actualizar empleado
- DELETE/empleados/:id ⟹ Eliminar empleado

Métodos HTTP

- **POST:** permite enviar datos para crear un recurso
- **GET:** permite obtener información
- **PUT:** permite actualizar datos existentes
- **DELETE:** permite eliminar información

Evidencias

- **Crear empleados**



- **Listar empleados**

GET http://localhost:4000/empleados/ Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1  {
2    "id": 4,
3    "nombre": "Andres",
4    "cargo": "Administrador"
5  }
6
```

Status: 200 OK Size: 198 Bytes Time: 6 ms

Response Headers 6 Cookies Results Docs

```
1  [
2    {
3      "id": 2,
4      "nombre": "Carlos",
5      "cargo": "Programador"
6    },
7    {
8      "id": 1,
9      "nombre": "Ana",
10     "cargo": "Diseñadora senior"
11   },
12   {
13     "id": 3,
14     "nombre": "Laura",
15     "cargo": "Gerente"
16   },
17   {
18     "id": 4,
19     "nombre": "Andres",
20     "cargo": "Administrador"
21   }
22 ]
```

GET http://localhost:4000/empleados/ Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1  {
2    "id": 4,
3    "nombre": "Andres",
4    "cargo": "Administrador"
5  }
6
```

Status: 200 OK Size: 147 Bytes Time: 6 ms

Response Headers 6 Cookies Results Docs

```
1  [
2    {
3      "id": 3,
4      "nombre": "Laura",
5      "cargo": "Gerente"
6    },
7    {
8      "id": 2,
9      "nombre": "Carlos",
10     "cargo": "Programador"
11   },
12   {
13     "id": 1,
14     "nombre": "Ana",
15     "cargo": "Diseñadora senior"
16   }
17 ]
```

- ## Buscar empleados

Welcome to Settings Sync

GET http://localhost:4000/empleados/4 Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1  {
2    "id": 4,
3    "nombre": "Andres",
4    "cargo": "Administrador"
5  }
6
```

Status: 200 OK Size: 50 Bytes Time: 5 ms

Response Headers 6 Cookies Results Docs

```
1  {
2    "id": 4,
3    "nombre": "Andres",
4    "cargo": "Administrador"
5  }
```

GET http://localhost:4000/empleados/2 Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1  {
2    "id": 4,
3    "nombre": "Andres",
4    "cargo": "Administrador"
5  }
6
```

Status: 200 OK Size: 48 Bytes Time: 4 ms

Response Headers 6 Cookies Results Docs

```
1  {
2    "id": 2,
3    "nombre": "Carlos",
4    "cargo": "Programador"
5  }
```

- ## Actualizar empleado

PUT http://localhost:4000/empleados/1 Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

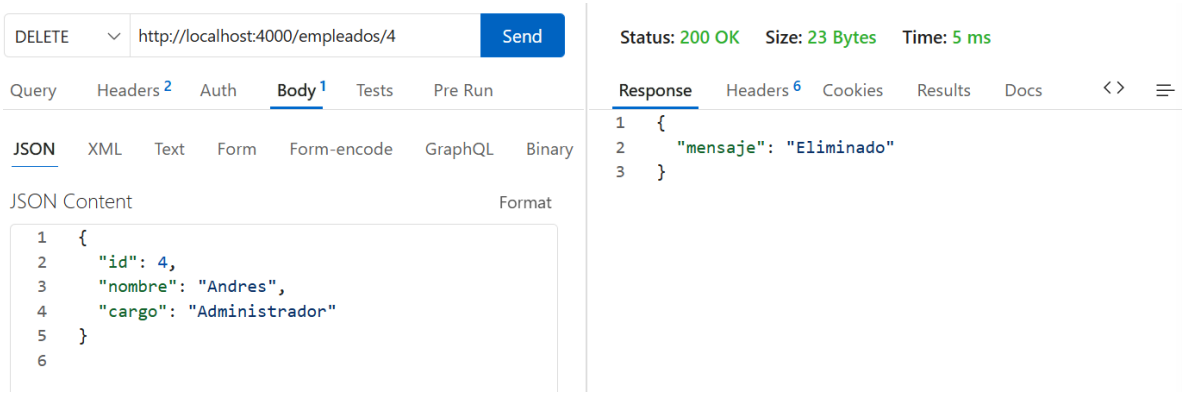
```
1  {
2    "id": 1,
3    "nombre": "Ana",
4    "cargo": "Diseñadora senior"
5  }
6
```

Status: 200 OK Size: 89 Bytes Time: 6 ms

Response Headers 6 Cookies Results Docs

```
1  {
2    "mensaje": "Actualizado",
3    "empleado": {
4      "id": 1,
5      "nombre": "Ana",
6      "cargo": "Diseñadora senior"
7    }
8  }
```

- **Eliminar empleado**



Conclusión

La API desarrollada cumple con las operaciones básicas CRUD, permitiendo gestionar empleados de forma eficiente. Se logro implementar correctamente el uso de métodos HTTP y rutas en Express.